

Technical Architecture Document

Swasthya Katha — Multilingual Visual Health Reader

Version: 1.0

Status: Draft for architecture approval

Date: 24 August 2026

1. Architecture objective

Build a maintainable, secure, accessible, multilingual application in which visual richness is layered on top of a reliable semantic learning system.

The architecture must make the correct path the easiest path:

- Content is structured and validated.
- Medical data is versioned.
- Text is always available.
- Visuals have fallback representations.
- WebGL is optional.
- Audio is synchronized through structured timing data.
- Staff workflows are permissioned.
- Reader state is explicit.

2. Architecture principles

1. Semantic HTML is the foundation.
2. SVG is the primary educational visual format.
3. CSS is the default page-motion engine.
4. WebGL is progressive enhancement.
5. Content is data, not hard-coded JSX.
6. Publication is versioned and reversible.
7. The public reader is separate from editorial administration.
8. Server-side authorization is mandatory.
9. Offline support is scoped and observable.
10. Every heavy feature has a lite fallback.

3. Proposed stack

Frontend

- Next.js App Router.
- React.
- TypeScript in strict mode.
- CSS Modules or controlled utility CSS.
- `next-intl` for locale-aware routing and messages.
- Motion or GSAP for timelines.
- React Three Fiber and Drei for selected WebGL scenes.
- Zod for runtime content validation.
- XState or a small explicit state machine for reader control.

Next.js App Router is designed for modern full-stack applications and route-level layouts and rendering. [web:12][web:14]

`next-intl` provides localized rendering, formatting, and locale-based routing support for Next.js applications. [web:34][web:37]

Backend

- Supabase PostgreSQL.
- Supabase Storage.
- Supabase Auth for staff and optional learners.
- Next.js route handlers/server actions.
- GitHub for source, schemas, and content review history.
- Google Cloud Run for optional media processing.

Quality and operations

- Playwright for browser tests.
- Vitest or Jest for unit tests.
- axe-based accessibility checks.
- ESLint and TypeScript.
- Dependency and secret scanning.
- Error monitoring.
- Performance monitoring.

4. System context

Reader device

- > CDN/Next.js application
- > Published content API
- > Public asset storage
- > Local cache and reader state

Staff browser

- > Authenticated admin routes
 - > Review API
- > Protected draft storage
- > Audit log
- > Publication service

Optional processing

- > Audio normalization
- > Transcript timing
- > SVG sanitization
- > Asset optimization

5. Application route structure

```
app/
├── [locale]/
│   ├── layout.tsx
│   ├── page.tsx
│   ├── reader/[bookSlug]/page.tsx
│   ├── reader/[bookSlug]/[chapterSlug]/page.tsx
│   ├── learn/[lessonSlug]/page.tsx
│   ├── help/page.tsx
│   ├── privacy/page.tsx
│   └── accessibility/page.tsx
├── api/
│   ├── publications/
│   ├── progress/
│   └── admin/
└── admin/
    ├── publications/
    ├── lessons/
    ├── reviews/
    └── releases/
```

6. Repository structure

```
src/
├── domain/
│   ├── publication/
│   ├── lesson/
│   ├── review/
│   └── user/
└── components/
```

```
|   ├── book/
|   ├── reader/
|   ├── scenes/
|   ├── audio/
|   ├── quizzes/
|   └── ui/
├── state/
├── accessibility/
├── i18n/
├── content/
├── security/
└── performance/
```

```
content/
├── en/
├── hi/
└── shared/
```

```
public/
├── illustrations/
├── audio/
├── fonts/
└── textures/
```

```
supabase/
├── migrations/
├── seed/
└── policies/
```

```
tests/
├── unit/
├── integration/
├── e2e/
├── accessibility/
└── visual/
```

7. Domain model

Publication

```
export type Publication = {
  id: string
  slug: string
  title: string
  status: "draft" | "review" | "published" | "withdrawn"
  version: string
  locales: string[]
  clinicalReviewDueAt: string
}
```

Lesson block

```
export type LessonBlock = {
  id: string
  type: "text" | "illustration" | "diagram" | "animation" | "quiz" | "action"
  order: number
  content: LocalizedText[]
  sceneId?: string
  narrationTrackId?: string
  accessibilityDescription?: LocalizedText[]
}
```

Visual scene

```
export type VisualScene = {
  id: string
  renderer: "svg" | "webgl" | "static"
  fallbackAsset?: string
  sceneDescription: string
  targetIds: string[]
}
```

Narration event

```
export type TimelineEvent = {
  id: string
  startMs: number
  endMs: number
  blockId: string
  textTokenIds: string[]
  sceneTargetIds: string[]
  action: "highlight" | "pulse" | "trace" | "show"
}
```

8. Database design

```
publications
chapters
lessons
lesson_blocks
lesson_translations
visual_scenes
scene_targets
narration_tracks
word_timings
highlight_sequences
quizzes
quiz_options
source_references
content_reviews
```

```
publication_versions
users
roles
user_progress
bookmarks
audit_events
technical_events
```

9. Content API

Public manifest

```
GET /api/publications/{slug}?locale=hi&version=latest
```

Server requirements:

- Validate locale.
- Return only published content.
- Resolve approved translation.
- Return immutable asset URLs.
- Include publication version.
- Exclude internal notes and draft metadata.

Lesson endpoint

```
GET /api/publications/{slug}/lessons/{lessonSlug}?locale=hi
```

Server requirements:

- Verify publication status.
- Validate response schema.
- Resolve fallback policy.
- Include accessibility descriptions.
- Exclude unapproved content.

10. Reader state architecture

```
CLOSED
-> OPENING
-> READY

READY
-> TURNING_FORWARD
-> TURNING_BACK
-> AUDIO_PLAYING
```

```
-> LINEAR_MODE
-> SETTINGS_OPEN
-> ERROR_FALLBACK

AUDIO_PLAYING
-> AUDIO_PAUSED
-> AUDIO_ENDED
-> VISUAL_HIGHLIGHTING

ERROR_FALLBACK
-> RETRY
-> LINEAR_MODE
```

The state machine must own:

- Current page.
- Current block.
- Current mode.
- Flip progress.
- Audio status.
- Active highlight.
- Motion setting.
- Rendering profile.
- Error state.

11. Magazine reader implementation

CSS foundation

- `transform-style: preserve-3d`.
- Spine-origin page rotation.
- CSS variables for index and turn progress.
- Layered shadows.
- Page thickness illusion.
- Pointer-events controlled by active page.

Interaction controller

- Pointer capture.
- Touch swipe.
- Keyboard navigation.
- Commit threshold.
- Velocity threshold.

- Snap-back state.
- Focus restoration.

WebGL enhancement

- Load only for active page or selected hero scene.
- Use a subdivided plane.
- Deform vertices according to turn progress.
- Apply page lighting and crease shading.
- Dispose resources after leaving the scene.
- Keep DOM text over or beside the canvas.

The referenced CSS flipbook demonstrates a useful minimal transform model, but production must add state control, accessibility, touch handling, and fallback behaviour. [web:11]

12. Visual scene architecture

```
SceneDefinition
-> target registry
-> renderer selection
-> animation timeline
-> accessibility description
-> static fallback
```

SVG target example:

```
<g id="testing-clinic" role="img" aria-label="Testing clinic">
  ...
</g>
```

The target registry should map IDs to:

- Human label.
- Description.
- Narration aliases.
- Focus order.
- Fallback text.

13. Audio synchronization

One locale-specific timeline controls:

- Audio playback.
- Transcript token highlighting.
- SVG target focus.

- Optional scene animation.
- Action-card reveal.

Rules:

- Pause all synchronized animation when audio pauses.
- Recalculate after seek.
- Restore correct state after page visibility changes.
- Never require audio for comprehension.

14. Rendering profiles

Enhanced

- WebGL page effects.
- Advanced shadows.
- High-resolution assets.
- Optional ambient transitions.

Balanced

- CSS page turns.
- Semantic SVG.
- Limited textures.
- No heavy particle systems.

Lite

- Linear reader.
- Static SVG or raster.
- Minimal motion.
- Deferred audio.
- No WebGL.

Profile selection uses:

- Device capability.
- Network information where available.
- User preference.
- Reduced-motion preference.
- Manual override.

15. Offline architecture

Cache:

- App shell.
- Publication manifest.
- Current lesson.
- Essential SVG and audio.
- Local settings.
- Local progress.

Do not cache:

- Staff drafts.
- Sensitive user data unnecessarily.
- Expired content without a stale indicator.

16. Performance budgets and evidence

Initial targets:

- First useful lesson appears within 3 seconds on the agreed 4G test profile.
- Optional WebGL does not block first content.
- Core lesson remains usable on the lite profile.
- Repeated page turns do not create unbounded memory growth.
- Audio loading does not freeze the reader.

Evidence:

- Lighthouse or equivalent report.
- Device recordings.
- Memory profile.
- Network-throttled test.
- WebGL failure test.

17. Observability

Track technical events:

- `reader_opened`.
- `lesson_loaded`.
- `audio_load_failed`.
- `visual_fallback_used`.
- `webgl_unavailable`.

- `publication_load_failed`.
- `offline_cache_miss`.
- `client_error`.

Do not attach sensitive reading topics or identity unless explicitly justified and approved.

18. Architecture acceptance criteria

- Content can be added without modifying reader components.
- A lesson works in linear mode without WebGL.
- A visual can be replaced by a static fallback.
- A new locale can be added without duplicated application logic.
- Publication versions are immutable.
- Draft content cannot be returned by public endpoints.
- Audio timing validates before publication.
- Reader state is testable without DOM scraping.
- Heavy assets are lazy-loaded.
- Architecture decisions are recorded.